

Roteiro do vídeo

Cerca de 16 minutos. Para cada parte: a fala, o comando que você digita e o que vai aparecer na tela.

Onde digitar: no **Azure Cloud Shell**. Entre em `portal.azure.com`, clique no ícone `>` no topo e escolha **Bash**. Ele já vem conectado na sua conta e tem tudo o que você precisa — não instale nada.

Antes de ligar a gravação

10 minutos de preparo

Passo A · Entrar no portal da Azure

1. Abra o navegador e digite `portal.azure.com`
2. Faça login com `RM564099@fiap.com.br` e a sua senha da FIAP
3. Se pedir para "Continuar conectado", pode clicar em **Sim**

Passo B · Achar o grupo de recursos

1. Na **barra de busca no topo** da página (onde está escrito "Pesquisar recursos, serviços e documentos"), digite: `rg-564099-dimdim`
2. Nos resultados, embaixo de **Grupos de recursos**, clique em `rg-564099-dimdim`
3. Abre uma tela com a lista dos **4 recursos**. É essa tela que você vai mostrar no vídeo — **deixe ela aberta**

Passo C · Abrir o Cloud Shell (o terminal)

1. No **canto superior direito** da página, ao lado do ícone de sino, tem um ícone `>`. Clique nele
2. Se aparecer a pergunta "Bash ou PowerShell", escolha **Bash**
3. Se pedir conta de armazenamento, escolha a opção mais simples que aparecer (qualquer uma serve)
4. O terminal abre na parte de baixo da tela. Você pode arrastar a borda para deixá-lo maior

Passo D · Deixar o projeto baixado

Clique dentro do terminal e cole os comandos abaixo (para colar no Cloud Shell use **Ctrl+Shift+V**, ou clique com o botão direito):

```
DIGITE
git clone https://github.com/natcsouza/dimdim-cp4-containers.git
cd dimdim-cp4-containers
```

Isso baixa os arquivos que você vai mostrar no vídeo (os Dockerfiles e o DDL).

Passo E · Conferir que a aplicação está no ar

```
DIGITE
curl http://564099-dimdim-app.eastus2.azurecontainer.io:8080/clientes
```

Se aparecer um texto com `Steves Jobs` e `Maria Fernandes`, está tudo funcionando.
Se der erro ou demorar muito, espere 1 minuto e tente de novo.

Passo F · Testar a senha do banco

DIGITE

```
mysql -h 564099-dimdim-db.eastus2.azurecontainer.io -u dimdim -p -D dimdim -e "SELECT * FROM cliente;"
```

Vai aparecer Enter password: — digite a senha do banco e aperte ENTER. **As letras não aparecem enquanto você digita**, isso é normal.

Se listar os dois clientes, a senha está certa e você já treinou o comando.

Passo G · Preparar a tela para gravar

1. Digite clear e aperte ENTER, para limpar o terminal
2. Aumente o tamanho da letra: segure **Ctrl** e aperte **+** umas 3 vezes
3. Feche as outras abas do navegador (WhatsApp, e-mail, qualquer coisa pessoal)
4. Deixe abertas só duas abas: o **portal da Azure** e o **GitHub** (github.com/natcsouza/dimdim-cp4-containers)

Passo H · Ligar a gravação de tela

No Windows, sem instalar nada:

1. Aperte **Windows + G** — abre a Barra de Jogos
2. Na janelinha **Capturar**, clique no botão redondo de **gravar** (ou aperte direto **Windows + Alt + R**)
3. Confira se o **microfone está ligado** — tem um ícone de microfone na mesma janelinha, ele não pode estar cortado
4. Para parar no final: **Windows + Alt + R** de novo
5. O vídeo é salvo em **Vídeos → Capturas**

Importante: a Barra de Jogos grava **a janela que estava aberta quando você começou**. Comece a gravação já com o navegador na frente, e não troque para outro programa durante o vídeo.

A senha do banco nunca é digitada dentro do comando. Nos SELECTs você usa -p sozinho: o MySQL pergunta a senha e **o que você digita não aparece na tela**. Senha visível no vídeo custa **20 pontos**.

Linha do tempo

TEMPO	PARTE
00:00	Abertura e apresentação
00:45	O desenho da arquitetura, no GitHub
01:45	Os recursos no portal da Azure
03:00	Os mesmos recursos pela linha de comando
04:30	As imagens no Container Registry
05:30	O Dockerfile da aplicação e a regra do não-root
07:30	O Dockerfile do banco e o DDL das tabelas
09:00	A estrutura da tabela no banco
09:45	CRUD: estado inicial
10:30	CRUD: CREATE
11:45	CRUD: READ
12:15	CRUD: UPDATE

13:15	CRUD: DELETE
14:15	Validação dos dados
14:45	Persistência no file share
16:00	Fechamento

Quem é você e o que vai mostrar

45s

FALE

Oi professor. Sou a Natalia Cristina de Souza, RM 564099, da turma 2TDSR. Esse é o primeiro checkpoint do segundo semestre, de imagem e containers em nuvem. O nosso projeto se chama DimDim: é uma API de cadastro de clientes, feita em Java com Spring Boot, e um banco de dados MySQL. Os dois estão containerizados, com as imagens registradas no Azure Container Registry, e rodando na Azure como Container Instances. Vou começar mostrando os recursos que eu criei.

00:45 · O desenho da arquitetura

1 minuto

Mostre o diagrama no GitHub

1min

FALE

Antes de entrar na Azure, queria mostrar o desenho da solução, que está no README do nosso repositório. São duas imagens guardadas no Container Registry. Cada uma vira um container rodando na Azure: um é o banco MySQL, o outro é a aplicação Java. A aplicação conversa com o banco pela porta 3306, e o banco grava os dados num file share da conta de armazenamento — por isso os dados não se perdem se o container for reiniciado. Esse segundo diagrama mostra o caminho da imagem: do Dockerfile até o container no ar.

O que fazer na tela:

1. Vá para a aba do GitHub que você deixou aberta
2. Role a página até o diagrama colorido embaixo de **Arquitetura**
3. Aponte com o mouse: primeiro as duas imagens no registry, depois os dois containers, e por último a conta de armazenamento embaixo
4. Role um pouco mais e mostre o segundo diagrama, **Do código até a nuvem**
5. Volte para a aba do portal da Azure

01:45 · Os recursos no portal

1 min 15

Mostre na tela, sem comando

1min15

FALE

Esse é o grupo de recursos do projeto, na região East US 2. Aqui dentro estão os quatro recursos: o Container Registry, que guarda as imagens; a conta de armazenamento, onde ficam os dados do banco de forma persistente; e as duas instâncias de container, uma do banco e uma da aplicação.

O que fazer na tela, um passo de cada vez:

1. Você já está na tela do grupo rg-564099-dimdim, com a lista dos 4 recursos
2. Mova o mouse devagar sobre cada nome da lista enquanto fala: acr564099dimdim (o registry), st564099dimdim (o armazenamento), aci-564099-dimdim-db (o banco) e aci-564099-dimdim-app (a aplicação)
3. Agora clique em st564099dimdim
4. No **menu da esquerda**, procure a seção **Armazenamento de dados** e clique em **Compartilhamentos de arquivos**
5. Clique em dimdim-mysql — é aqui que os dados do banco ficam guardados
6. Para voltar, clique em rg-564099-dimdim na trilha de navegação no topo da página

O grupo de recursos

30s

FALE

Agora os mesmos recursos pela linha de comando, com a Azure CLI. Foi assim que tudo foi criado: os scripts estão no repositório do GitHub.

DIGITE

```
az group show --name rg-564099-dimdim --output table
```

Aparece: o nome do grupo e a região.

Os quatro recursos

30s

FALE

Esses são os quatro recursos, os mesmos que apareciam no portal.

DIGITE

```
az resource list --resource-group rg-564099-dimdim --output table
```

Aparece: o registry, a conta de armazenamento e os dois containers.

Os containers no ar

30s

FALE

E aqui os dois containers rodando, cada um com o seu endereço público e a imagem que está usando. A aplicação na porta 8080 e o banco na porta 3306.

DIGITE

```
az container list --resource-group rg-564099-dimdim --output table
```

Aparece: os dois com status Succeeded, o IP e a porta.

04:30 · As imagens no Container Registry

1 minuto

As duas imagens

40s

FALE

Essas são as duas imagens que eu construí e registrei no Container Registry: uma do banco e uma da aplicação. Repare que as duas começam com o meu RM, 564099, que é como o enunciado pediu para nomear.

DIGITE

```
az acr repository list --name acr564099dimdim --output table
```

Aparece: 564099-dimdim-app e 564099-dimdim-db.

FALE

E essa é a versão da imagem da aplicação, a 1.0.

DIGITE

```
az acr repository show-tags --name acr564099dimdim --repository 564099-dimdim-app --output table
```

Build em dois estágios e usuário sem privilégio

1min15

FALE

Esse é o Dockerfile da aplicação. Ele tem dois estágios. No primeiro eu uso a imagem do Maven para compilar o projeto Java e gerar o jar. No segundo, eu copio só o jar pronto para uma imagem que tem apenas o Java de execução, o JRE. Isso deixa a imagem final bem menor, porque o Maven e o código-fonte não vão junto. E aqui embaixo está a parte que o enunciado exige: eu crio um usuário chamado dimdim e uso o comando USER. A partir dessa linha, nada roda como root dentro do container.

DIGITE

```
cat app/Dockerfile
```

Aponte na tela: a linha FROM maven ... AS build, depois FROM eclipse-temurin:17-jre-alpine, e por fim adduser -S dimdim e USER dimdim.

Provando que não é root

45s

FALE

E não basta estar escrito no Dockerfile: vou provar no container que está rodando agora. Vou executar o comando id dentro dele.

DIGITE

```
az container exec --resource-group rg-564099-dimdim --name aci-564099-dimdim-app --exec-command "id"
```

Aparece: uid=100(dimdim) gid=101(dimdim). **Fale:** "uid 100, usuário dimdim — não é o uid zero, que seria o root."

07:30 · O banco e o DDL

1 min 30

O Dockerfile do banco

40s

FALE

Esse é o Dockerfile do banco. Ele parte da imagem oficial do MySQL 8 e copia o script de DDL para a pasta de inicialização do MySQL. Quando o container sobe pela primeira vez, o MySQL executa esse script sozinho e cria a tabela.

DIGITE

```
cat db/Dockerfile
```

O DDL das tabelas

50s

FALE

E esse é o DDL. Ele cria o banco dimdim e a tabela cliente, com a chave primária no id, o CPF como campo único, e uma restrição que não deixa o saldo ser negativo. Esse arquivo está no repositório, como o enunciado pede.

DIGITE

```
cat db/init/01-ddl.sql
```

Aponte: PRIMARY KEY, UNIQUE no CPF e o CHECK (vl_saldo >= 0).

09:00 · A estrutura no banco

45 segundos

SHOW TABLES e DESCRIBE

45s

FALE

Agora vou me conectar no banco que está rodando na Azure e mostrar que a tabela foi criada exatamente como está no DDL.

DIGITE

```
mysql -h 564099-dimdim-db.eastus2.azurecontainer.io -u dimdim -p -D dimdim -e "SHOW TABLES; DESCRIBE cliente;"
```

Ele pede a senha — digite e aperte ENTER, não aparece na tela. Depois aparece a tabela cliente e as cinco colunas.

09:45 · CRUD: estado inicial

45 segundos

Pela API e pelo banco

45s

FALE

Agora a demonstração do CRUD. Primeiro o estado inicial: a API responde neste endereço público e devolve dois clientes.

DIGITE

```
curl http://564099-dimdim-app.eastus2.azurecontainer.io:8080/clientes
```

FALE

E os mesmos dois registros direto no banco, com um SELECT. A partir de agora, depois de cada operação eu volto aqui para provar no banco.

DIGITE

```
mysql -h 564099-dimdim-db.eastus2.azurecontainer.io -u dimdim -p -D dimdim -e "SELECT * FROM cliente;"
```


POST na API

45s

FALE

A primeira operação é o CREATE. Vou inserir um cliente novo mandando um POST para a API, com os dados em JSON no corpo da requisição.

DIGITE

```
curl -X POST http://564099-dimdim-app.eastus2.azurecontainer.io:8080/clientes -H "Content-Type: application/json" -d '{"nome":"Joao Carlos Menk","cpf":"99988877766","email":"joao.menk@dimdim.com.br","saldo":8750.50}'
```

Aparece: o JSON do cliente criado, com "id": 3. **Anote esse número** — é ele que você usa nos próximos passos.

SELECT confirmando

30s

FALE

E aqui está a evidência no banco: a linha com o id 3 foi gravada na tabela. A inserção não ficou só na resposta da API, ela chegou no MySQL.

DIGITE

```
mysql -h 564099-dimdim-db.eastus2.azurecontainer.io -u dimdim -p -D dimdim -e "SELECT * FROM cliente;"
```

Aparece: três linhas agora.

11:45 • READ

30 segundos

GET por id

30s

FALE

A segunda operação é o READ. Vou buscar esse mesmo cliente pela API, passando o id na URL.

DIGITE

```
curl http://564099-dimdim-app.eastus2.azurecontainer.io:8080/clientes/3
```

Aparece: o JSON só desse cliente.

PUT na API

35s

FALE

A terceira operação é o UPDATE. Vou alterar dois campos desse cliente: o e-mail, que passa de dimdim ponto com ponto br para fiap ponto com ponto br, e o saldo, que vai de oito mil setecentos e cinquenta para doze mil e trezentos.

DIGITE

```
curl -X PUT http://564099-dimdim-app.eastus2.azurecontainer.io:8080/clientes/3 -H "Content-Type: application/json" -d '{"nome":"Joao Carlos Menk","cpf":"99988877766","email":"joao.menk@fiap.com.br","saldo":12300.00}'
```

SELECT confirmando

25s

FALE

E no banco os dois campos realmente mudaram: o e-mail agora é fiap e o saldo é doze mil e trezentos.

DIGITE

```
mysql -h 564099-dimdim-db.eastus2.azurecontainer.io -u dimdim -p -D dimdim -e "SELECT * FROM cliente;"
```

13:15 • DELETE

1 minuto

DELETE na API

30s

FALE

A quarta e última operação é o DELETE. Vou remover esse cliente. A API responde com o código 204, que significa que apagou com sucesso e não tem conteúdo para devolver.

DIGITE

```
curl -i -X DELETE http://564099-dimdim-app.eastus2.azurecontainer.io:8080/clientes/3
```

Aparece: HTTP/1.1 204 na primeira linha.

SELECT confirmando

30s

FALE

E no banco o registro sumiu. Voltamos aos dois clientes do começo. Com isso as quatro operações do CRUD foram demonstradas, e cada uma delas foi confirmada com um SELECT direto no banco de dados.

DIGITE

```
mysql -h 564099-dimdim-db.eastus2.azurecontainer.io -u dimdim -p -D dimdim -e "SELECT * FROM cliente;"
```

A API recusa dado inválido

30s

FALE

Antes de encerrar, uma coisa a mais: a API valida o que recebe. Se eu tentar cadastrar um cliente sem nome, com e-mail inválido e saldo negativo, ela recusa com o código 400 e não grava nada no banco.

DIGITE

```
curl -i -X POST http://564099-dimdim-app.eastus2.azurecontainer.io:8080/clientes -H "Content-Type: application/json" -d '{"nome":"","cpf":"11111111111","email":"nao-e-email","saldo":-5}'
```

Aparece: HTTP/1.1 400.

14:45 · Persistência

1 min 15

Reiniciando o container do banco

45s

FALE

Por último, a persistência dos dados. Os dados do MySQL não ficam dentro do container: eles ficam no file share da conta de armazenamento, que está montado na pasta var, lib, mysql. Para provar isso, vou reiniciar o container do banco.

DIGITE

```
az container restart --resource-group rg-564099-dimdim --name aci-564099-dimdim-db
```

Demora cerca de um minuto. Enquanto espera, explique que o container está sendo derrubado e recriado, e que se os dados estivessem dentro dele seriam perdidos agora.

SELECT depois do restart

30s

FALE

O container reiniciou e os dados continuam lá, intactos. Isso prova que a persistência está funcionando pela conta de armazenamento, como o enunciado pediu.

DIGITE

```
mysql -h 564099-dimdim-db.eastus2.azurecontainer.io -u dimdim -p -D dimdim -e "SELECT * FROM cliente;"
```

Aparece: os mesmos dois clientes.

Encerrando

30s

FALE

Para fechar: todos os recursos foram criados pela Azure CLI, e os scripts estão no repositório do GitHub, junto com os dois Dockerfiles, o DDL das tabelas, os arquivos JSON de teste e o README com o passo a passo do build e do push das imagens. Era isso, professor. Obrigada.

O que fazer na tela:

1. Troque para a aba do GitHub que você deixou aberta
2. Clique na pasta scripts e mostre os arquivos de 01 a 06 — são os comandos da Azure CLI
3. Volte e clique em db, depois em init, e mostre o 01-ddl.sql
4. Volte e clique em json — são os arquivos de teste
5. Volte para a página inicial do repositório, onde aparece o README

Conferência final

- ☐ Vídeo em 720p ou mais, com sua voz do começo ao fim
- ☐ Começa mostrando os recursos criados na Azure
- ☐ As quatro operações do CRUD, cada uma seguida do SELECT no banco
- ☐ Aparece a prova de que a aplicação não roda como root
- ☐ Nenhuma senha apareceu escrita na tela
- ☐ Vídeo no YouTube como "**Não listado**" — privado significa nota zero
- ☐ Link do vídeo dentro do PDF da folha de rosto
- ☐ PDF enviado no Teams por você, que é a representante do grupo

Se errar no meio: não precisa recomeçar tudo. Continue de onde parou e corte depois, ou simplesmente refaça a frase — o professor não desconta por isso.